

Introducción a la arquitectura

Definición de arquitectura de software

Es el set de estructuras (vistas*) del sistema que comprenden:

- Elementos de SW **
- Propiedades externamente visibles***
- Relaciones que existen entre ellos ****

Solo se ocupa del costado público de las interfaces. Los detalles de implementación no son arquitectónicos.

* No hay una sola vista que permite entender el sistema en forma apropiada. Abstracción permite entender la complejidad. Las vistas son selectivas (mostramos algunas cosas, ignoramos otras).

** Módulos, Objetos, Componentes, Subsistemas, Elementos de compilación, Procesos.

*** Servicios que provee el elemento, características de performance, reacción ante fallos.

**** Calls, Send-data-to, Synchronizes-with, Uses, Depends, Instantiates.

Todo sistema o programa tiene una arquitectura.

¿Por qué es importante?

- Porque constituye un vehículo de comunicación entre stakeholders.
- Provee un lenguaje común para que todos puedan hablar y comprender.
- Provee un blueprint técnico para la construcción, modificación o análisis del sistema.
- Es el producto de las primeras decisiones de diseño.
- Las decisiones iniciales de arquitectura permitirán o impedirán casi todos los atributos de calidad del sistema.
- Es una abstracción del sistema que es reutilizable y transferible.
- Constituye un modelo relativamente chico de la estructura del sistema y la forma en que sus componentes trabajan en conjunto.

Estilo arquitectónico

Es una descripción de elementos y tipos de relaciones junto con un set de restricciones acerca de cómo pueden ser utilizados.

Son sistemas complejos que exhiben o contienen múltiples estilos al mismo tiempo.

Características de un estilo arquitectónico

Características

- Definen un set de elementos (por ejemplo: componentes, proceso, artefactos, módulos, etc.).
- Definen un set de tipo de conectores o mecanismo de interacción entre elementos (por ejemplo: call, event, pipe).
- Topología (cómo están distribuidos los elementos).
- Una serie de restricciones sobre esa topología y comportamiento de esos elementos (por ejemplo, en un repo de datos no está permitido modificar valores guardados; estilo pipelines es acíclico, etc.).
- Descripción informal de costos y beneficios del estilo, el **trade-off** (por ejemplo: pipe-and-filter favorece reutilización por encima de performance).

Viewtypes

Se selecciona dependiendo de qué utilidad da para el sistema.

Tiene 3 grupos de vistas:

Module (Vistas a nivel estático;
divisiones lógicas del sistema)

Decomposition

Uses

Layered

Class/generalization

Componente and Connectors

(A nivel ejecución)

Process

Concurrency

Shared-Data

Client-Server

Allocation

(Distribuciones, implementaciones, etc.)

Deployment

Implementation

Work Assignment

Rol del arquitecto

El rol del arquitecto es traducir las necesidades de negocio y requerimientos de usuarios en arquitecturas de alta calidad que cumplan con los objetivos.

- ✓ Definir cuál es el **problema**, cuáles son los **drivers** del negocio que participan, **escenarios arquitectónicos de relevancia** y cuáles son los **stakeholders**.
- ✓ Identificar **atributos de calidad** necesarios.
- ✓ Proveer un **diseño**, elegir **tecnologías**, trabajar en conjunto con el equipo de desarrollo para **asegurar** que se cumplan los **objetivos**.

El arquitecto es un puente entre los requerimientos del negocio y el sistema a construir.

Tipos de stakeholders e intereses



Cliente

Clientes, funcionalidad, lifetime, time-to-market, flexibilidad.



Usuario

Facilidad de uso, velocidad, disponibilidad.



Administradores

Configuración de sistema, administración de usuarios, seguridad, detección de intrusiones, backup, recovery y rebuild.



Managers

Amortizar costos de desarrollo, asegurar objetivos estratégicos, costos bajos, cumplimiento de plazos.



Developers

Lenguaje, tecnología.

¿Qué hace a una buena arquitectura?

Ley de Conway:

Las organizaciones que diseñan sistemas están limitadas a producir diseños que copian la estructura de comunicación de sus organizaciones.

Fuente: Conway, M. E. (1968). How do committees invent? *Datamation* 14(4), 28-31.

¿Qué es la calidad en el software?

Definición:

El grado en el cual un software posee una combinación deseada de atributos.

Fuente: IEEE Standard for a Software-Quality Metrics Methodology. IEEE Std 1061-1992. 1992. Institute of Electrical and Electronics Engineers, New York, NY.

¿Qué es un atributo de calidad o requerimiento no funcional?

Definición: propiedad medible o testeable que es utilizada para indicar qué tan bien satisface el sistema las necesidades de los interesados.

“Este puede ser tal vez el tema más importante de la arquitectura de software, el más amplio y de seguro el más complejo. Los atributos de calidad son características no funcionales que se desean garantizar del sistema que se está diseñando, a menudo son conocidos como requerimientos no funcionales y son, en definitiva, el principal insumo de un arquitecto”.

Ejemplos de atributos de calidad

Performance

Indica respuesta de un sistema a acciones específicas en un intervalo de tiempo. Se mide en latencia (tiempo de respuesta) y throughput (cantidad de eventos que pueden suceder en un tiempo dado).

Reliability

Habilidad de continuar operando de forma esperada en un tiempo dado. Se mide la probabilidad de que el sistema no falle en un intervalo de tiempo.

Availability

Define proporción de tiempo en el que el sistema está funcional y trabajando. Se puede medir como porcentaje de downtime sobre tiempo total de funcionamiento.

Usability

Intuición de uso, diseño orientado al uso y usuario, accesibilidad, buena UX.

Ejemplos de atributos de calidad

Reusability

Probabilidad de reutilización de componentes con poco o ningún cambio por otros componentes o escenarios.

Maintainability

Habilidad para poder manejar cambios con facilidad, en diferentes niveles y por diferentes razones.

Interoperability

Habilidad de operar comunicándose e intercambiando información con otros sistemas externos. Consideraciones clave: protocolos de comunicación, interfaces, formatos de datos.

.

Manageability

Facilidad de gestión y monitoreo a través de exposición de elementos para sistemas de monitoreo, para tuning de performance y debugging.

Ejemplos de atributos de calidad

Scalability

Habilidad de manejar aumento de carga sin afectar performance del sistema o habilidad de ser expandido fácilmente.

Supportability

Habilidad de proveer información útil para identificar y resolver problemas si el sistema no funciona correctamente. (Ejemplo: diagnóstico).

Security

Capacidad de reducir chances de acciones maliciosas o accidentales que puedan afectar el funcionamiento del sistema y/o la pérdida de información.

Testability

Definición de qué tan aptos son los componentes o sistemas para crear criterios de test y ejecutar tests para probar estos criterios.

Buena arquitectura

No hay buenas o malas arquitecturas. Hay buenas arquitecturas que solucionan un problema específico en un contexto específico.

Se evalúan por si cumplen o no sus objetivos en el momento de la definición o revisión.



© Universidad de Palermo

Prohibida la reproducción total o parcial de imágenes y textos.